UML Diagrams

Smart Traffic Complaint and Monitoring System

Project team: Arya, Ansh, Arnav, Avyukt

Technology baseline: MERN (MongoDB, Express.js, React, Node.js), Python FastAPI, PyTorch / ResNet18

Implementation update: citizen image upload, server-side AI inference integration, and CI workflow added. Diagrams reflect the current implementation described in the project work.

1. Use-Case Diagram

The system has two primary actors. Citizens register/log in, submit complaints with optional road images, view their own complaints, and track status. Administrators view and filter complaints, set priority, assign/modify status, and view dashboard summaries.

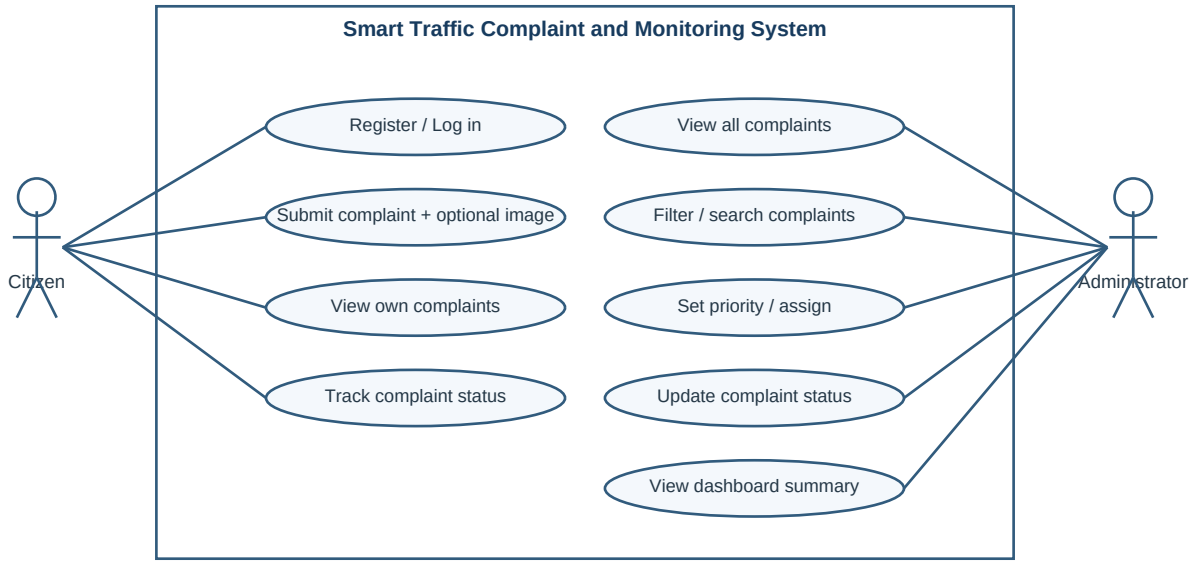
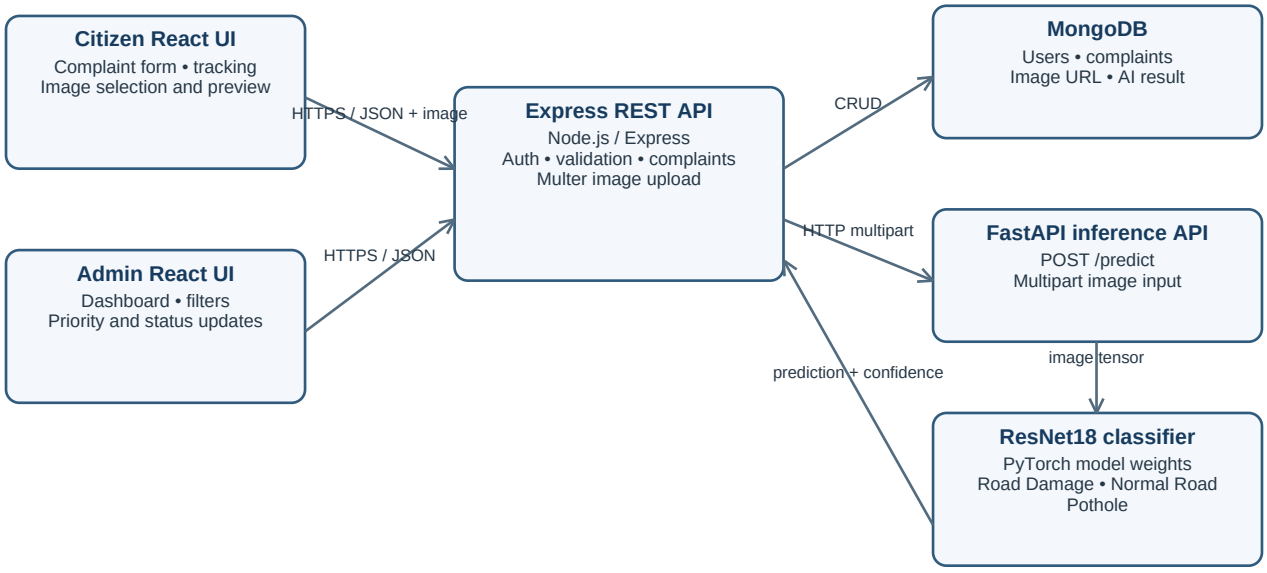


Figure 1. Citizen and administrator use cases, including optional image submission.

2. Component Diagram

The React citizen and administrator interfaces communicate with the Express REST API. The API handles authentication, authorization, complaint validation and file upload. Complaint metadata and prediction output are stored in MongoDB. The Express API calls the separate FastAPI inference service, which runs the ResNet18 model.



AI inference is a separate Python service; complaint submission is retained if prediction fails.

Figure 2. Updated architecture including image upload and AI inference components.

3. Sequence Diagram — Submit Complaint with AI Classification

The citizen submits complaint details and an optional image. Express validates the request, stores the complaint and image reference, then calls FastAPI for inference when an image is present. The prediction result is saved with the complaint. If inference fails, the complaint remains saved and the AI status records failure rather than losing the complaint.

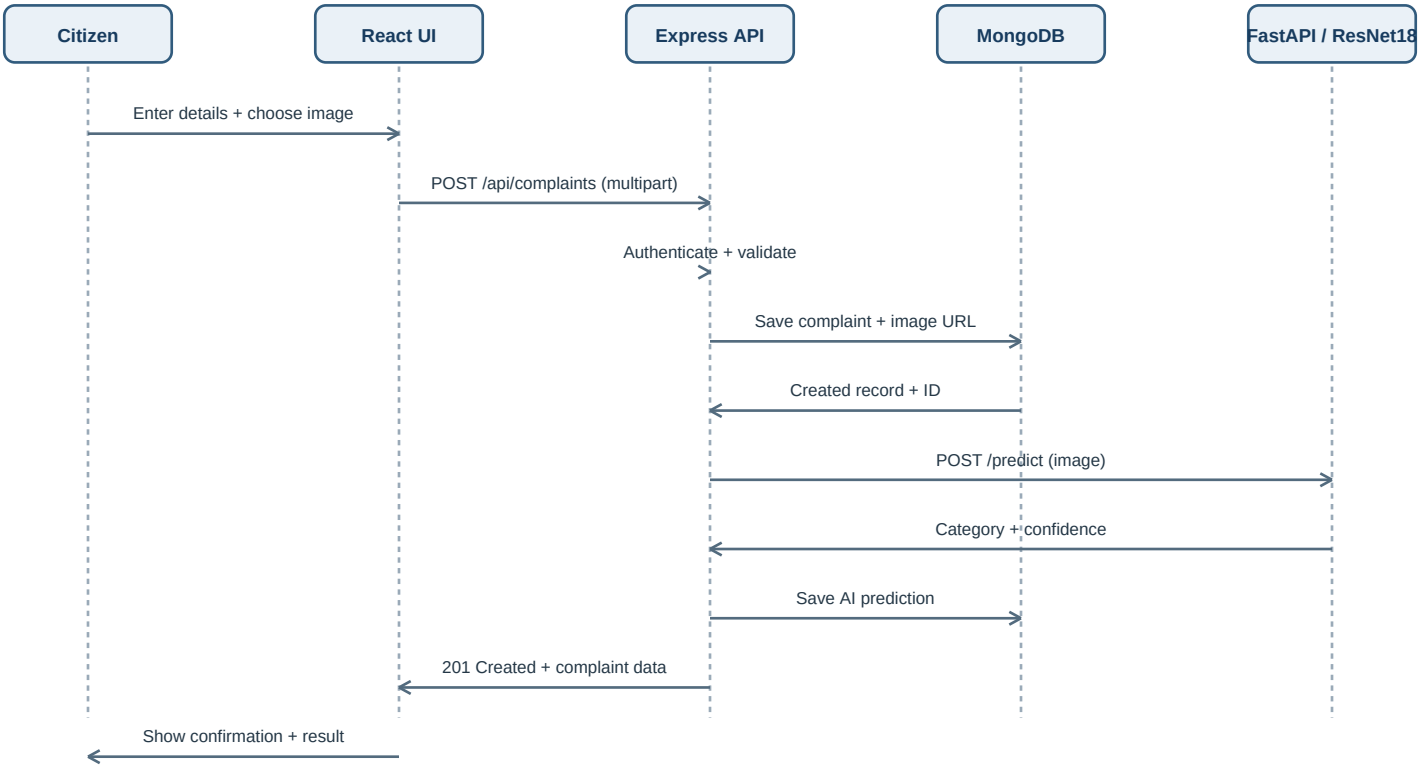


Figure 3. Complaint submission and optional AI classification sequence.

4. Sequence Diagram — Administrator Updates Complaint Status

The administrator selects a complaint and new status in the React admin interface. The Express API verifies the administrator role and validates the status before updating the MongoDB complaint record. The updated status is returned to the interface.

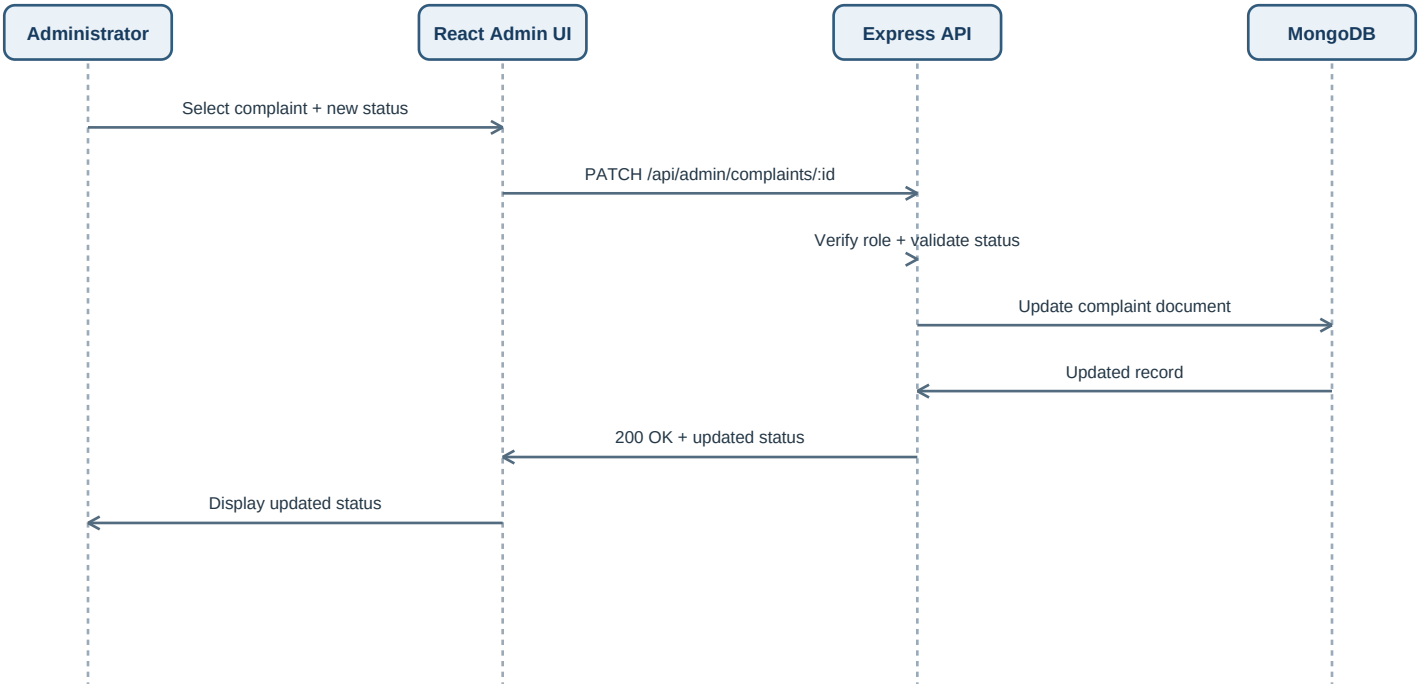


Figure 4. Administrator status update sequence.

5. Implementation Notes

• Image upload is handled by Multer in the Express backend; uploaded files are served from the server uploads path.
• FastAPI exposes health and prediction endpoints. The classifier uses the trained ResNet18 weights and returns category, confidence and class probabilities.
• The application maps classifier outputs to the complaint categories Road Damage, Normal Road and Pothole.
• GitHub Actions workflow **TrafficCare CI** runs backend tests and the frontend production build on pushes and pull requests targeting main.
• The CI workflow does not currently run an automated end-to-end test of the FastAPI model or a deployment.